ROADWATCH

Design | Project Assignment #3

University of Nevada, Reno

Department: Computer Science & Engineering

Course: CS 425 Software Engineering

Team Number: 19

Team Name: RoadWatch

Team Members: Christopher Thrasher, Vansh Sachdeva, Noe Rios, Victor Rosales

Instructors: Erin Keith, Zach Estreito

External Advisor(s): Travis Truhill - Director of Maintenance and Operations, City of Reno

Date: October 27, 2026

Table of Contents

2. Abstract **3**
3. Introduction **3**
4. Architecture and Structural Design **4**
4.1 System-Level Architecture 4
4.2 Subsystems, Classes, or Modules 6
4.3 Data Design 8
5. Detailed Design **10**
6. Initial Hardware Design **13**
7. User Interface Design **14**
7.1 Overview and Flow 14
7.2 Interface Snapshots 15
7.3 Usability Considerations 23
7.4 Link to Implementation 23
8. Version Control and Software Management System **24**
9. Contributions of Team Members **24**

2. Abstract

RoadWatch is a pothole detection and monitoring system intended to help municipalities organize roadway inspection information to support planning maintenance. The system processes uploaded smartphone video recordings using computer vision to identify potholes and associates the detection with GPS coordinates and a timestamp. Each detection is stored in a PostgreSQL database with its respective information, including an image and confidence score. A web-based dashboard presents detections as selectable markers on an interactive map, allowing any user and municipal staff to examine individual potholes. RoadWatch establishes a foundation for reviewing and tracking pothole detections, with additional filtering and maintenance management features planned as the project develops.

3. Introduction

RoadWatch is a pothole detection and monitoring system designed to support municipal roadway inspection and maintenance planning by automating roadway inspection. The system uses computer vision to identify potholes in recorded roadway footage. Each detection is associated with geographical coordinates and timestamps, all stored in a database and presented through an interactive web dashboard. By organizing the pothole records, RoadWatch aims to reduce reliance on manual pothole identification and provide useful information on roadway conditions.

Building on PA-2, the design translates the system's functional and non-functional requirements into a proposed software architecture and detailed component design. The design separates the key components into their own service: computer vision, detection data management, backend services, database storage, web interface, and a mobile application. A modularized structure helps improve maintainability by allowing individual components to be modified without requiring a system overhaul or changing the entire system.

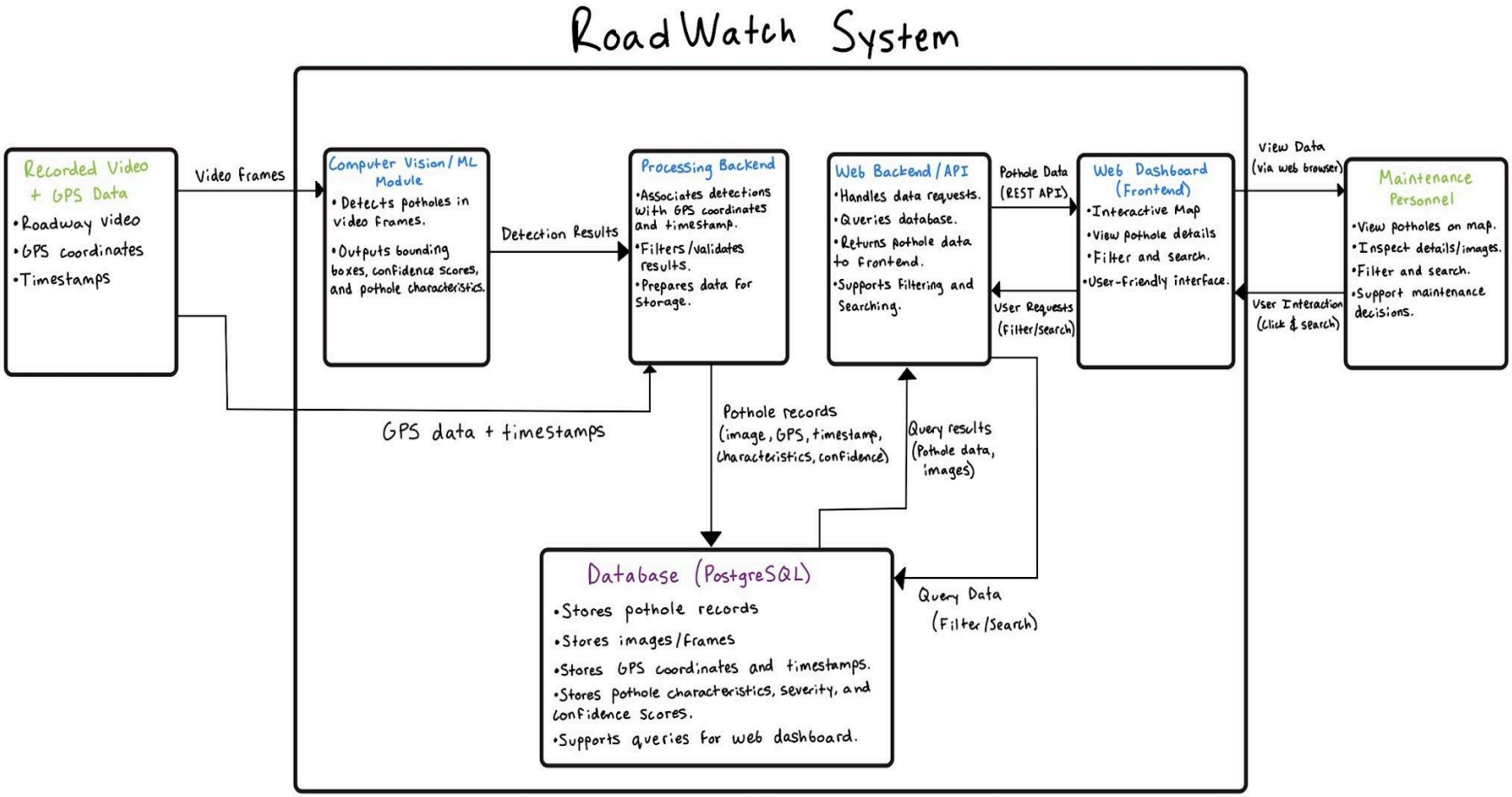
The project's scope and priorities have been refined, allowing for RoadWatch's system-level architecture, such as software modules, database design, relationships between videos, GPS information, pothole detections, and associated images, to be presented in a detailed workflow diagram. User interface designs for recording roadway footage, submitting roadway footage for processing, viewing detections on a map, and inspecting individual detection records have been created for a visual representation of the idea. The idea of how video footage is processed additionally changed, moving from having to connect to an external device such as a computer for analysis to now being sent to the servers for processing.

The team also has a clearer understanding of how RoadWatch could eventually be deployed. Rather than requiring dedicated inspection vehicles, the system will process footage collected by

cameras on vehicles already traveling city streets, allowing roadway conditions to be monitored with less manual effort.

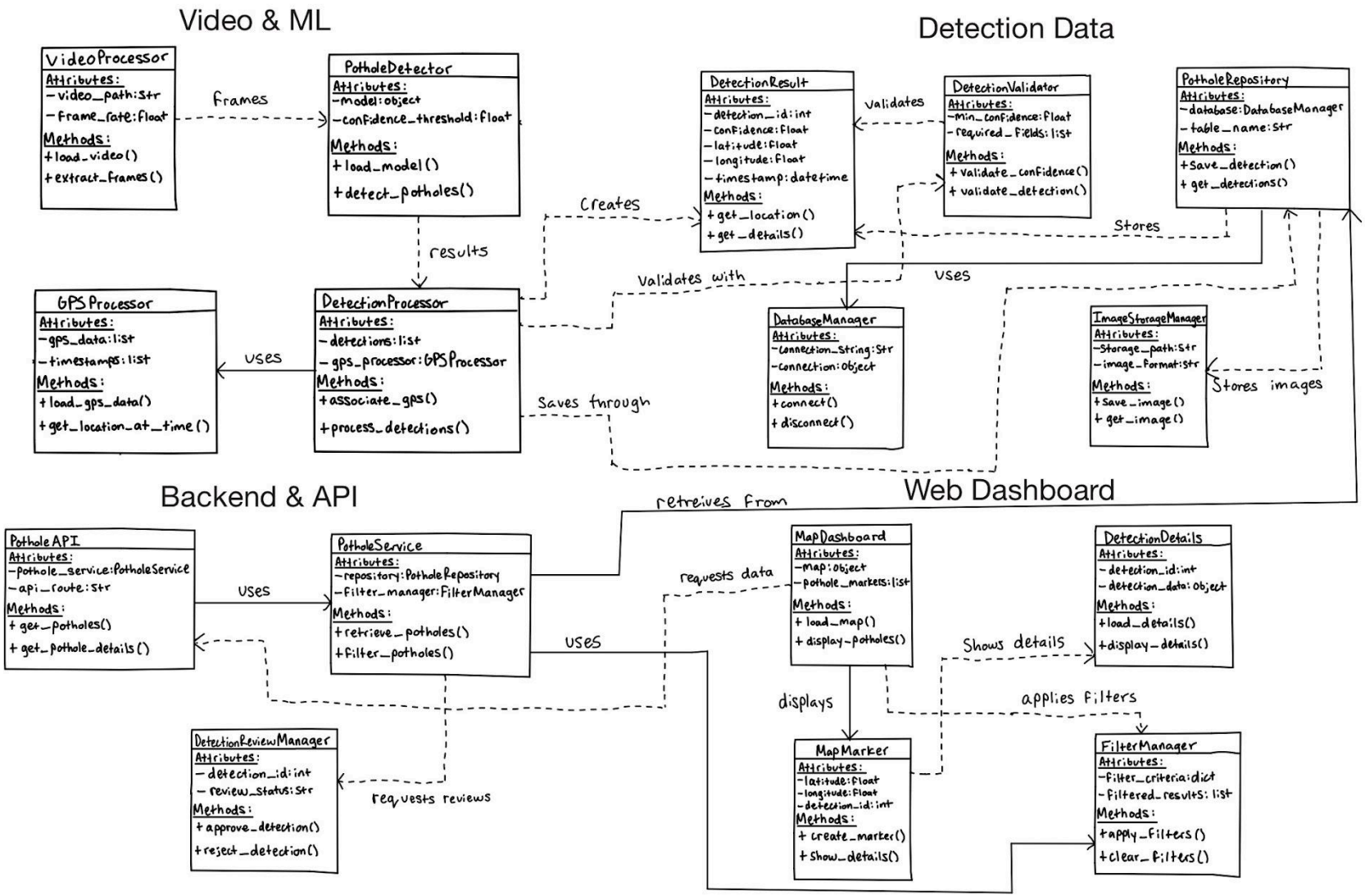
4. Architecture and Structural Design

4.1 System-Level Architecture



RoadWatch follows a layered architectural pattern that separates the computer vision module, backend processing, database storage, and web dashboard into different components. This architecture allows each component to be developed, tested, and maintained independently, making the system easier to modify as requirements change. It also supports scalability and maintainability by allowing individual components to be improved without requiring major changes to the rest of the system.

4.2 Subsystems, Classes, or Modules



Class Responsibilities and Requirement Mapping

Class	**Responsibility**	**PA2 IDs**
VideoProcessor	Loads recorded video and extracts frames.	FR-1, FR-2
GPSProcessor	Loads GPS data and matches locations to timestamps.	FR-3, FR-4
PotholeDetector	Uses computer vision to identify potholes and confidence scores.	FR-2, FR-7
DetectionProcessor	Processes detections and associates GPS and timestamps.	FR-3, FR-4
DetectionResult	Represents a detection and its stored information.	FR-3, FR-4, FR-5, FR-7
DetectionValidator	Checks detection information before storage.	FR-5
PotholeRepository	Saves and retrieves detection records.	FR-5, FR-7
DatabaseManager	Manages the PostgreSQL database connection.	FR-5
ImageStorageManager	Saves and retrieves pothole images.	FR-6
PotholeAPI	Handles dashboard requests for pothole data.	FR-8, FR-9
PotholeService	Retrieves and filters pothole information.	FR-8, FR-9, FR-12
DetectionReviewManager	Handles approval and rejection of detections.	FR-13
MapDashboard	Displays the interactive pothole map.	FR-8
DetectionDetails	Displays information about individual detections.	FR-9

MapMarker	Represents potholes as selectable map markers.	FR-8, FR-9
FilterManager	Applies and clears detection filters.	FR-12

RoadWatch is divided into separate subsystems for video processing and computer vision, detection data management, backend services, and the web dashboard. This separation allows changes to one part of the system without requiring major changes to the others. For example, the pothole detection model can be improved without changing how detection records are stored in the database or displayed on the map. Separating these responsibilities makes the system easier to maintain, test, and expand as new features are added.

4.3 Data Design

Entity	**Attributes**	**Primary Key**	**Foreign Key**
Video	video_id, file_name, upload_date, frame_rate	video_id	None
GPSRecord	gps_id, video_id, latitude, longitude, timestamp	gps_id	video_id
Detection	detection_id, video_id, gps_id, timestamp, confidence_score, status, latitude, longitude, severity, created_at, updated_at	detection_id	video_id, gps_id
DetectionImage	image_id, detection_id, image_path, capture_timestamp	image_id	detection_id
Review	review_id, detection_id,	review_id	detection_id

	review_status, review_date, notes, reviewer_id		
DetectionStatusHistory	history_id, detection_id, previous_status, new_status, change_date	history_id	detection_id

A Video can contain multiple GPS records and pothole detections. Each Detection is associated with a Video and can reference a GPSRecord containing its location information. A Detection can also have associated images, review records, and status history entries. These relationships allow RoadWatch to organize and retrieve pothole information while keeping related data connected.

The database design supports data integrity (NFR-3) by using primary and foreign keys to maintain relationships between pothole detections and their associated records. It also supports scalability (NFR-7) by separating information into related tables, allowing additional detections to be stored without changing the overall database structure. Storing detection information in PostgreSQL also supports reliability (NFR-2) by keeping records available after the application restarts.

5. Detailed Design

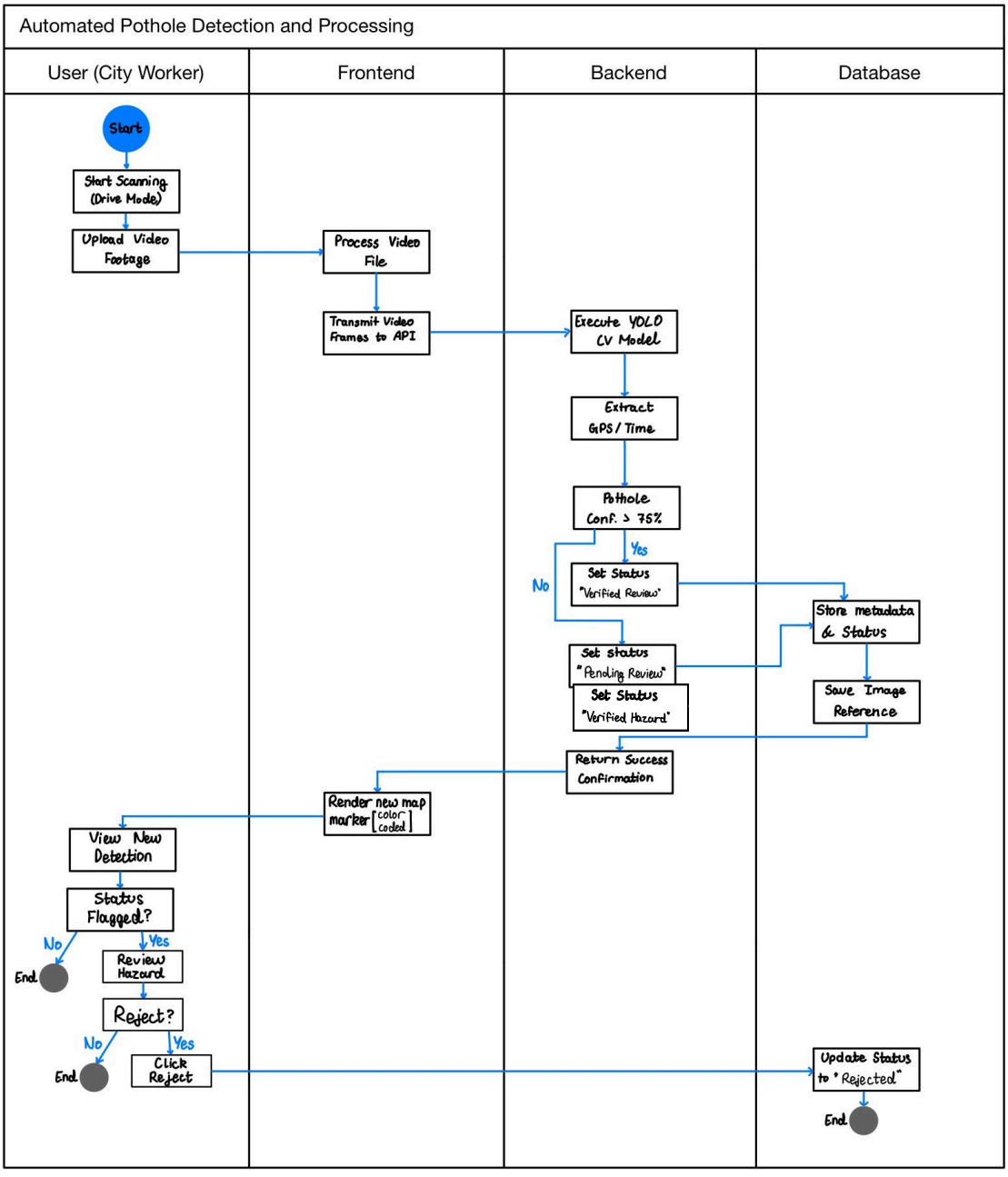


Diagram type: Activity Diagram
Referenced requirements: FR-1, FR-2, FR-3, FR-4, FR-5, FR-7, FR-13
Implementation link: VideoProcessor, GPSProcessor, PotholeProcessor, DetectionProcessor, PotholeRepository, DetectionResult, DetectionReviewManager.
Test Preview: Success when a video frame is processed, a pothole is detected with a >75% confidence score, GPS data is extracted, and the result is either saved to the database as “Verified Hazard” or flagged as “Pending Review”.

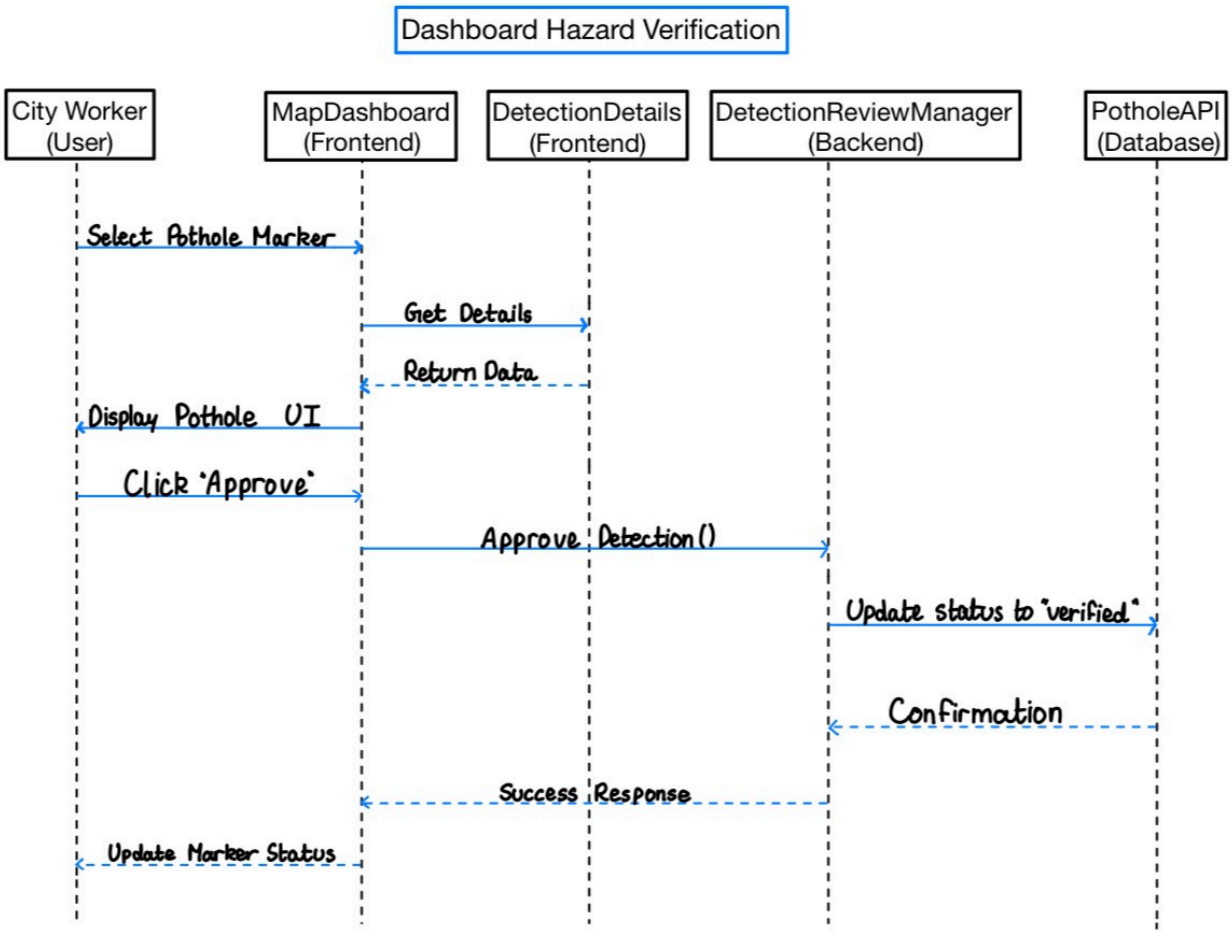


Diagram type: Sequence Diagram
Referenced requirements: FR-9, FR-13, FR-14
Implementation link: MapDashboard, DetectionDetails, DetectionReviewManager, PotholeAPI
Test preview: Success when an authorized user selects a pothole marker, views its details, and clicks approve, resulting in an updated status in the database.

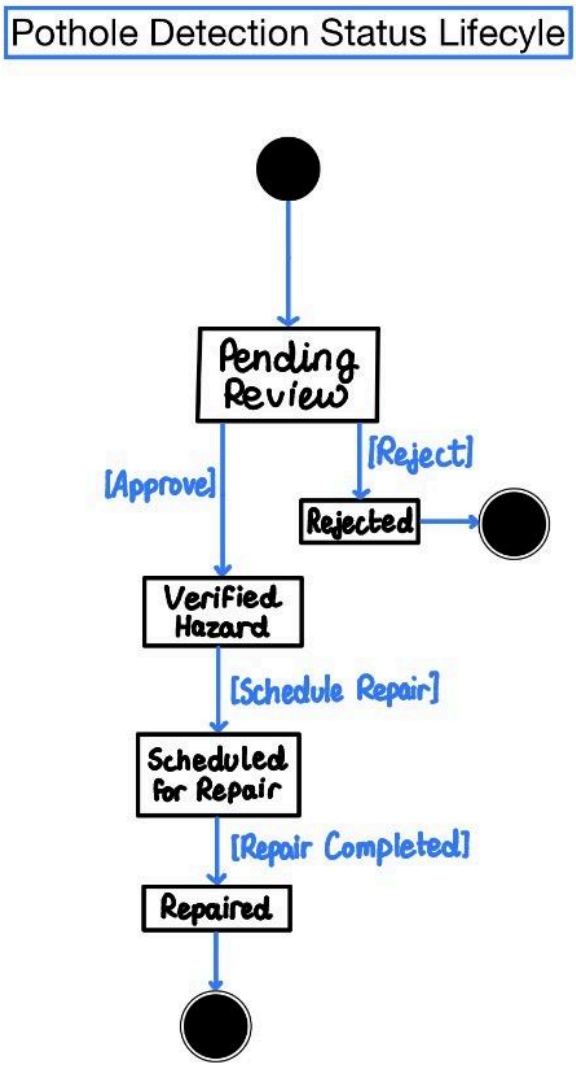


Diagram type: State diagram
Referenced requirements: FR-13, FR-14
Implementation link: DetectionReviewManager, PotholeAPI
Test preview: Success when a detection correctly transitions from “Pending Review” to “Verified Hazard” upon user approval, and updates to “Scheduled” when a work order is generated.

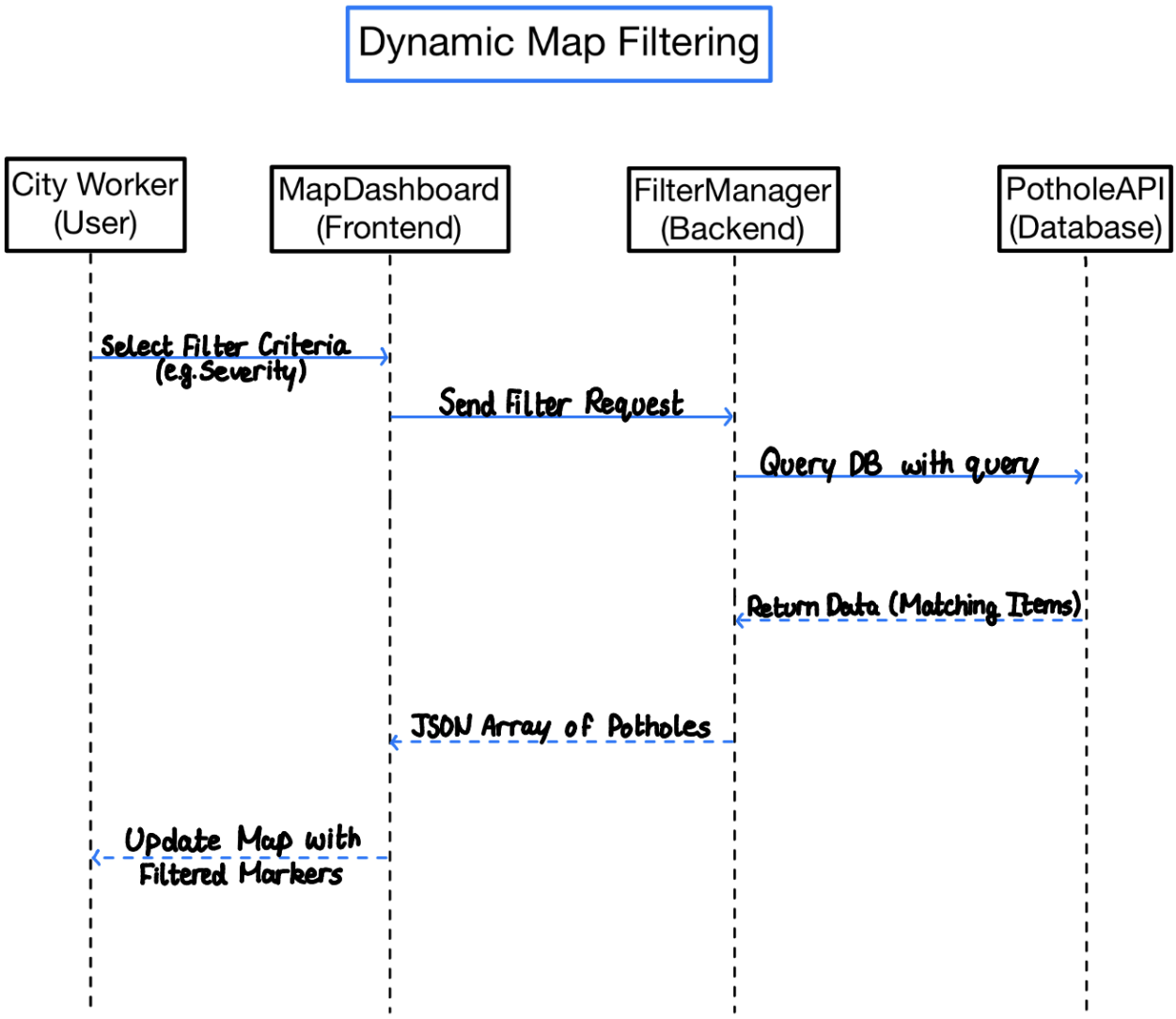


Diagram type: Sequence Diagram
Referenced requirements: FR-12, UC10
Implementation link: MapDashboard, FilterManager, PotholeAPI
Test preview: Success when a user applies a filter, the frontend requests filtered data from the API, and the map updates to display only the matching pothole markers.

6. Initial Hardware Design

Our Project is not a hardware project. The only hardware that will be used for this project will be smartphones that the city workers use on a daily basis, thus not applicable.

7. User Interface Design

7.1 Overview and Flow

RoadWatch's user interface is designed to help data collectors record roadway footage and help municipal staff inspect pothole detections through a straightforward web dashboard. The main goal is to make collecting, finding, and reviewing roadway information understandable without requiring technical knowledge.

Users begin by recording roadway footage with corresponding GPS and timestamp data, then submit the recording for offline processing. After processing finishes, municipal staff can view detections on an interactive map and select individual markers to inspect images, locations, timestamps, and confidence scores. Planned Level 2 features extend this workflow through filtering, detection review, and maintenance status tracking. Recording screens emphasize clear camera and GPS indicators, while dashboard screens use consistent navigation and clearly labeled controls. Throughout the workflow, the interface communicates processing progress, missing information, successful actions, and errors.

7.2 Interface Snapshots

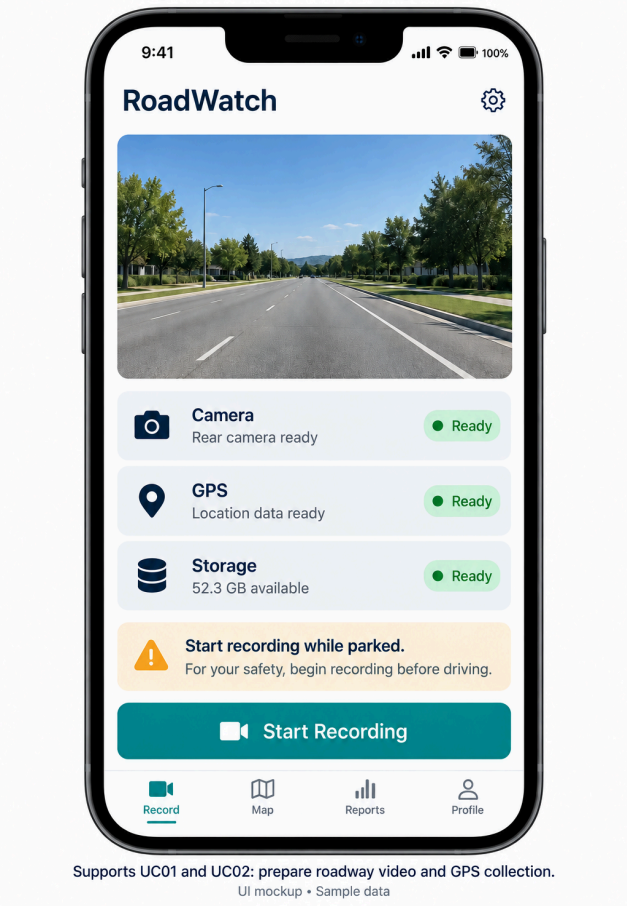


Figure 7.2.1: Recording Setup. This screen allows the data collector to check camera and GPS availability before starting a roadway recording while parked. It supports UC01 and UC02 by preparing the collection of footage and corresponding location data.

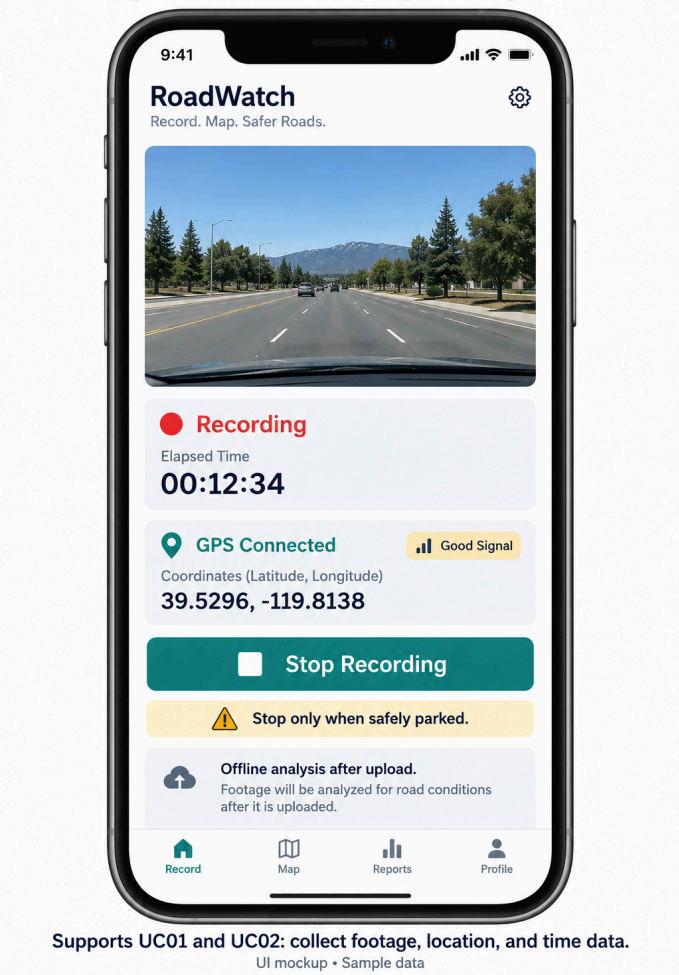


Figure 7.2.2: Recording in Progress. This screen displays recording activity, elapsed time, and GPS availability while roadway footage is collected for later processing. It supports UC01 and UC02 and is designed to avoid requiring interaction while driving.

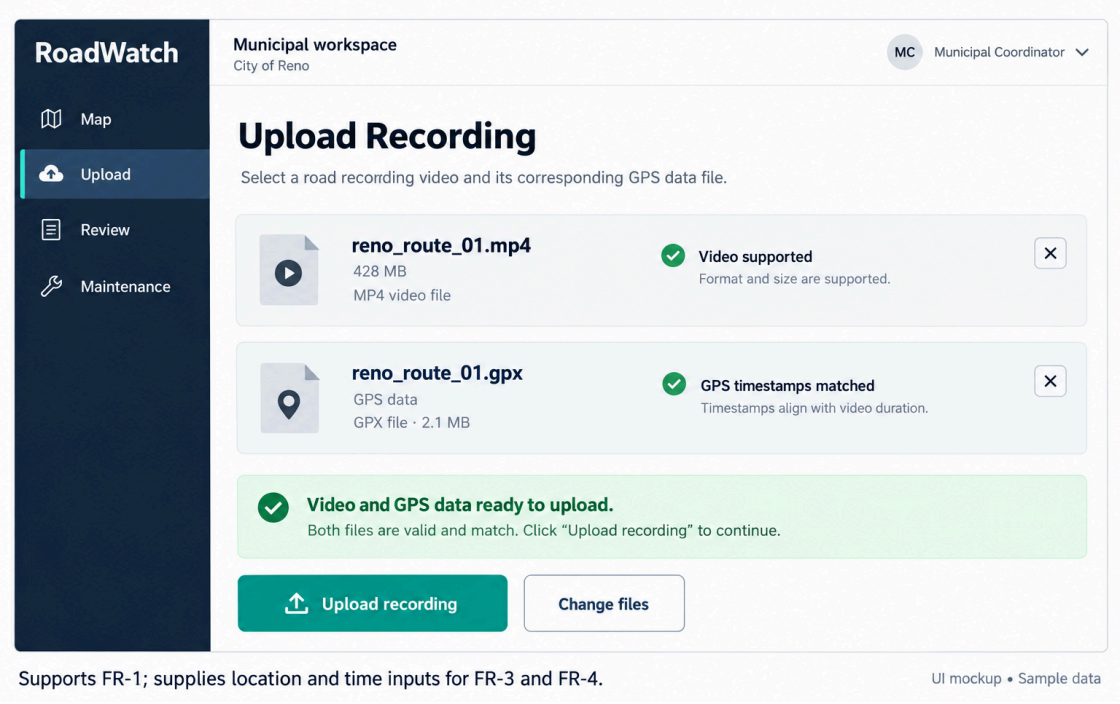


Figure 7.2.3: Upload Recording. This screen allows users to submit recorded roadway footage and its associated GPS data for processing, with feedback identifying missing or invalid inputs. It supports FR-1 and supplies the location and time inputs needed for FR-3 and FR-4.

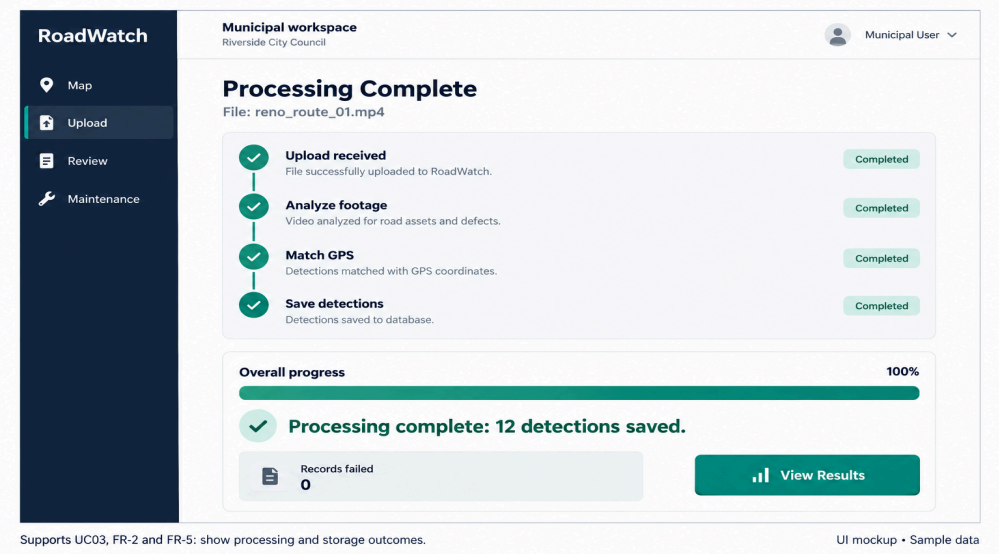


Figure 7.2.4: Processing Status. This screen communicates the processing state and displays the outcome once analysis finishes, including successful completion with zero detections or a processing failure. It supports UC03 and provides user feedback for the processing and storage operations described in FR-2 and FR-5.

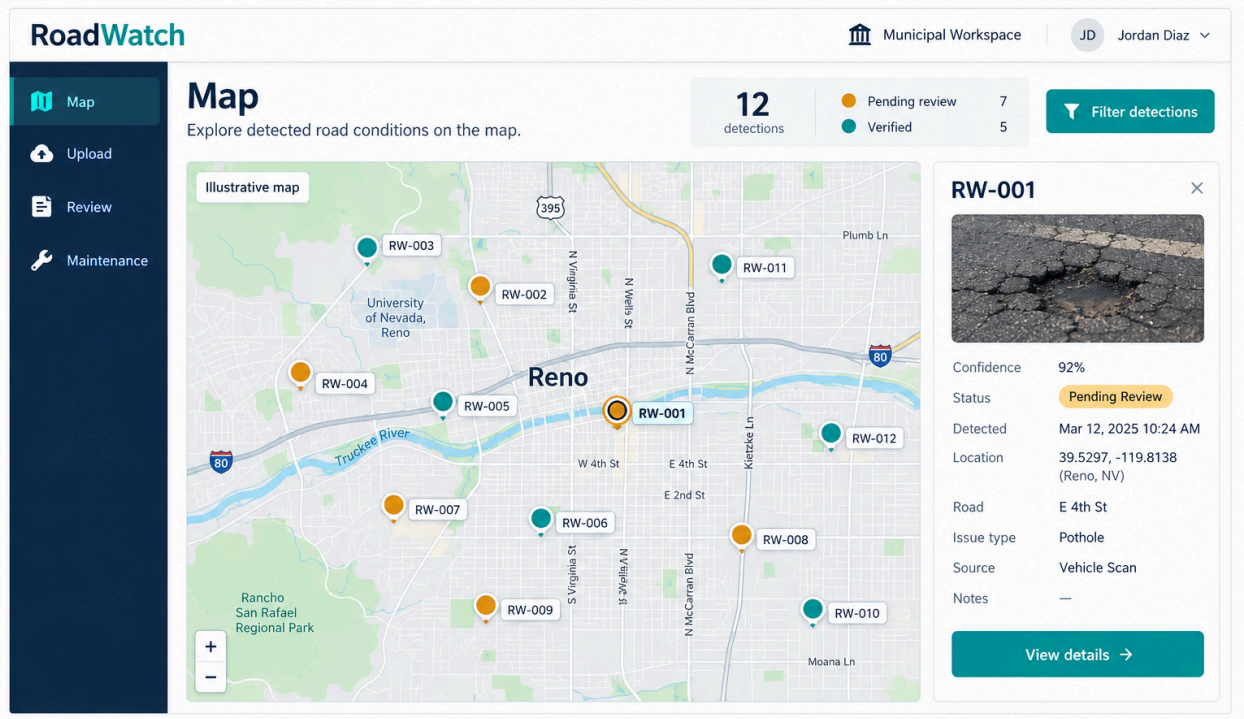


Figure 7.2.5: Map Dashboard. This screen displays stored detections with valid coordinates as selectable markers on an interactive map. It supports FR-8 and UC08 by allowing users to explore detections geographically.

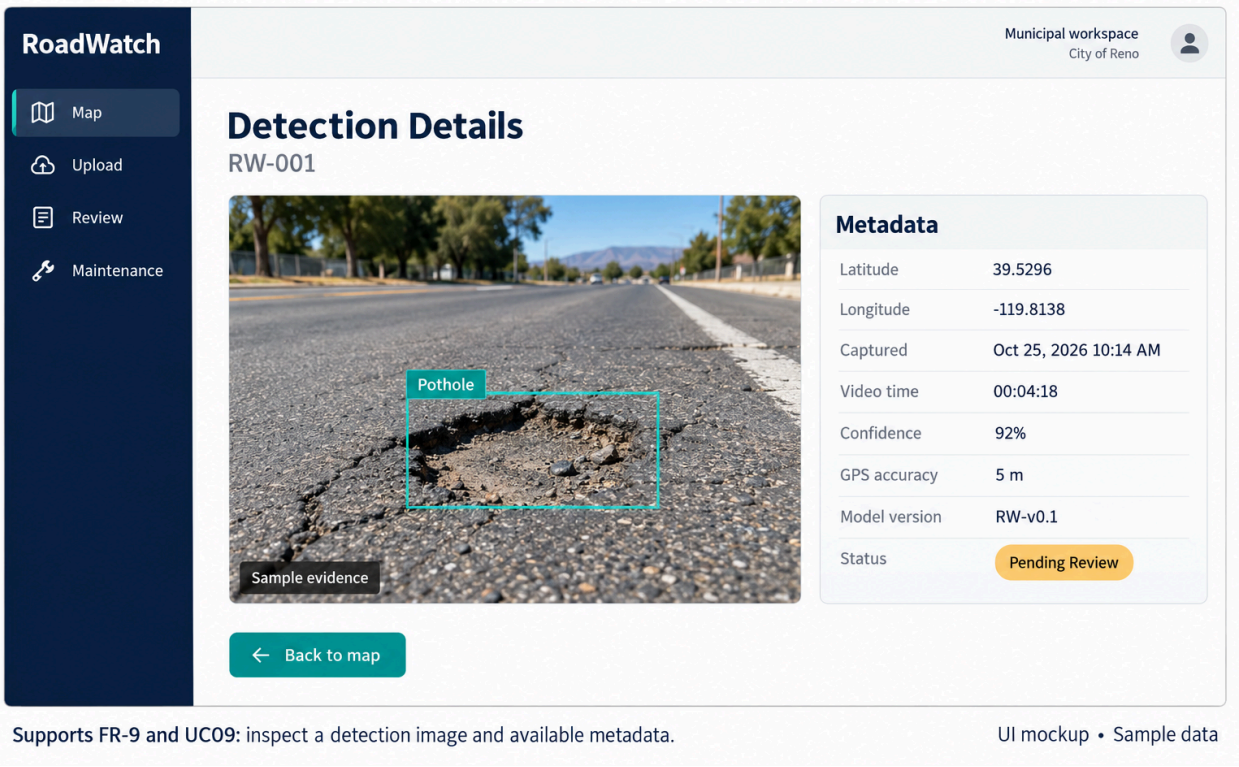


Figure 7.2.6: Detection Details. This screen displays the selected detection's image, location, timestamp, confidence score, and other available metadata. It supports FR-9 and UC09 by allowing users to inspect the evidence associated with an individual detection.

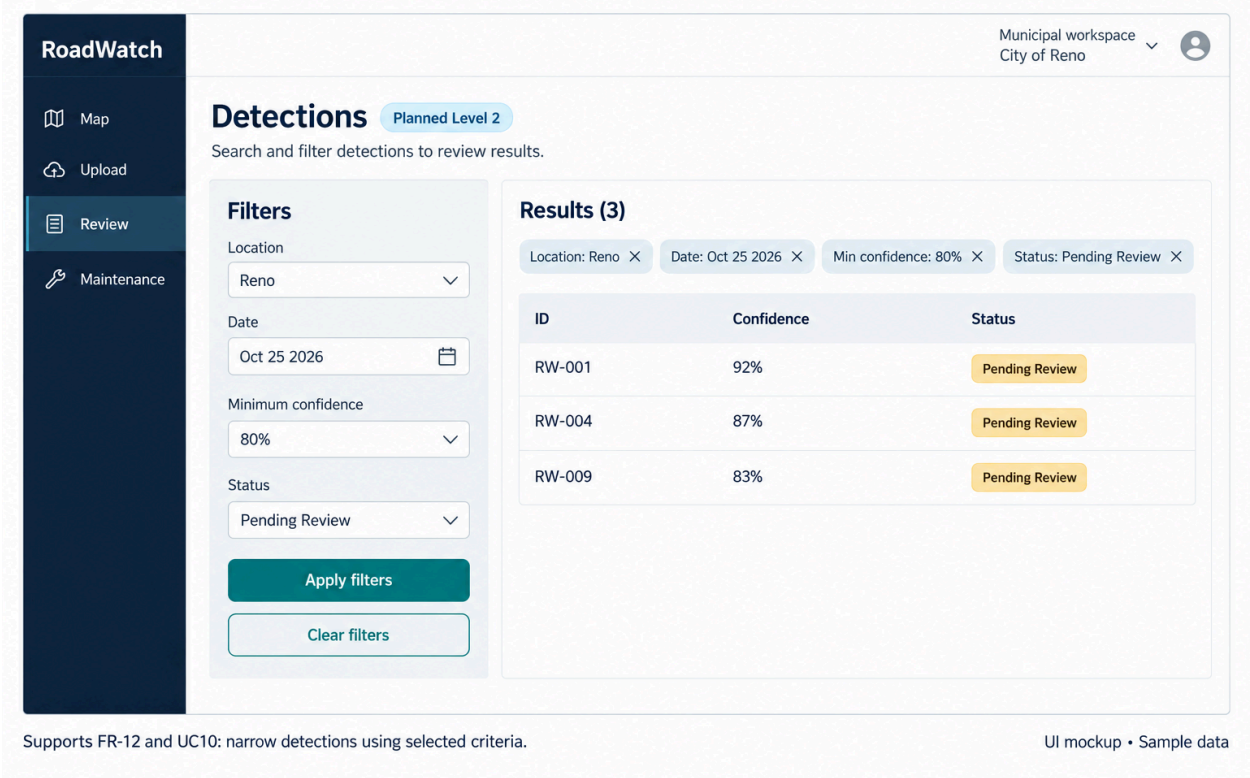


Figure 7.2.7: Search and Filters. This planned Level 2 screen allows users to narrow displayed detections by available criteria such as location, date, confidence, and status. It supports FR-12 and UC10 and provides feedback showing the number of matching results.

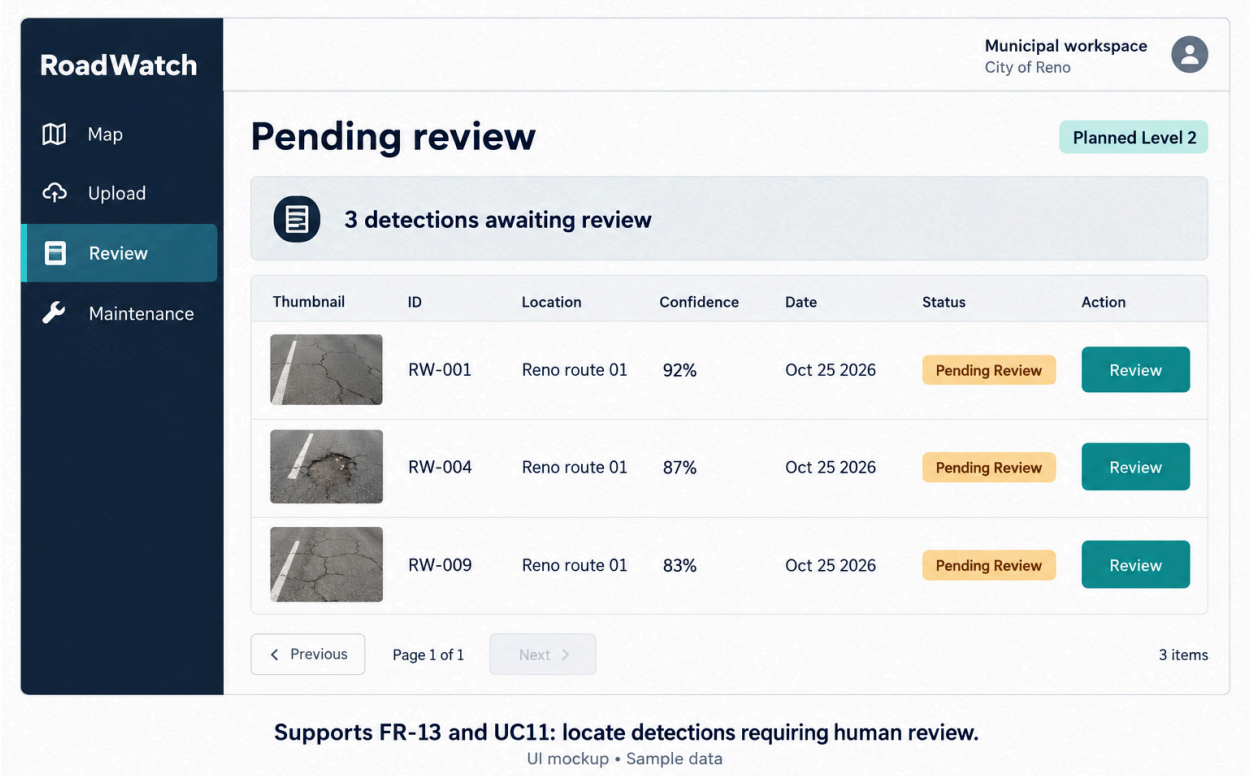


Figure 7.2.8: Review Queue. This planned Level 2 screen organizes detections awaiting human review and provides access to each detection's evidence. It supports FR-13 and UC11 by helping municipal staff locate and inspect records requiring verification.

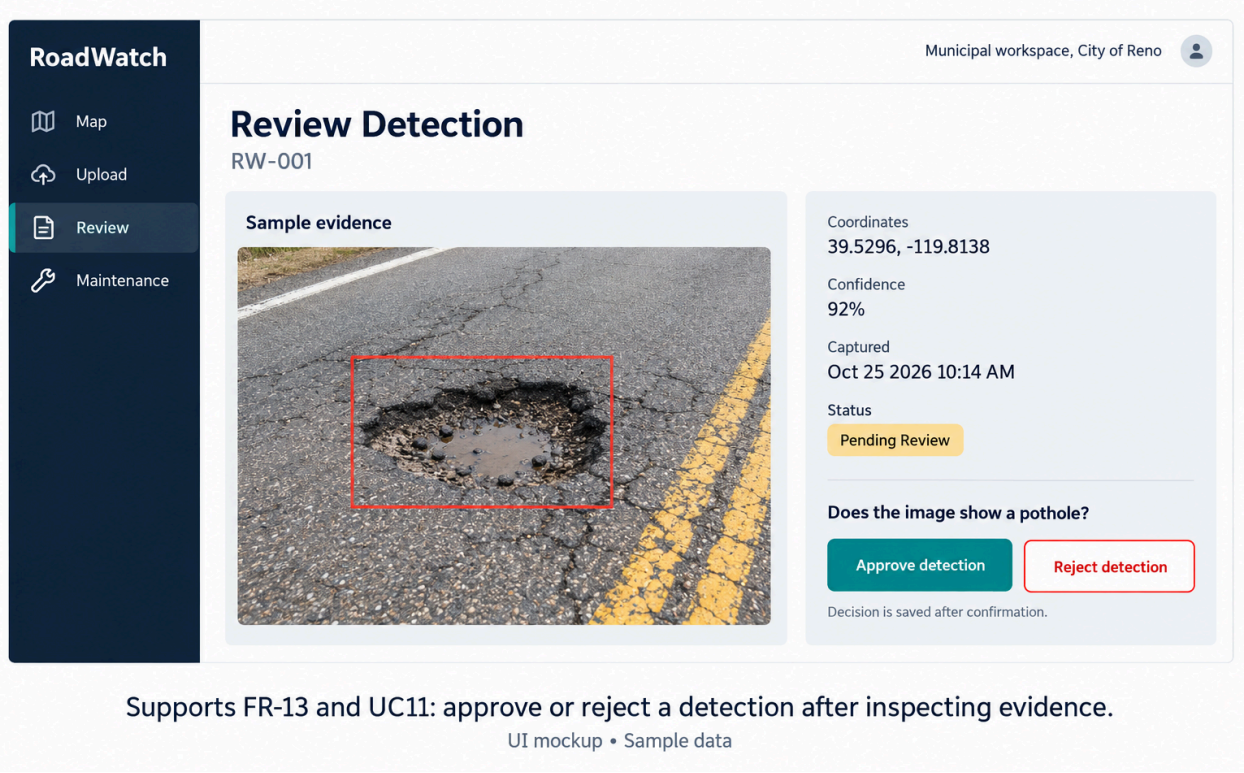


Figure 7.2.9: Review Detection. This planned Level 2 screen allows municipal staff to inspect detection evidence and approve or reject the result. It supports FR-13 and UC11 and displays confirmation after the review decision has been saved.

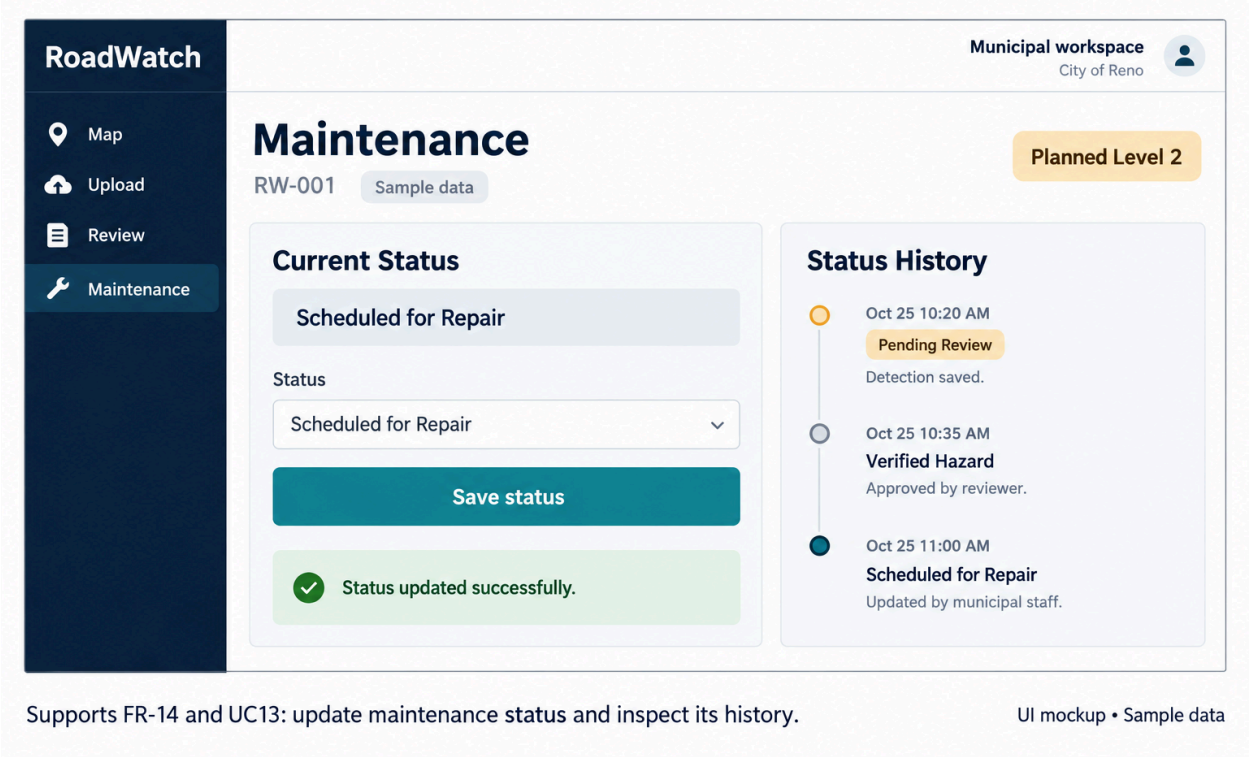


Figure 7.2.10: Maintenance Status and History. This planned Level 2 screen allows municipal staff to update a verified detection's maintenance status and view previous changes. It supports FR-14 and UC13 through status tracking, including Scheduled for Repair and Repaired.

7.3 Usability Considerations

RoadWatch will use consistent navigation, readable text, and clearly labeled buttons so first-time users can locate pothole information without developer assistance, supporting NFR-4. Status indicators will combine text or icons with contrasting colors so users do not have to rely on color alone. The interface will provide clear feedback for loading, empty results, missing information, errors, and successful actions, with layouts suited to mobile recording and desktop map review. We will conduct informal usability testing by asking classmates unfamiliar with the interface to open the dashboard, select a pothole marker, and find its details without assistance, then use their feedback to improve the design.

7.4 Link to Implementation

The web dashboard will use MapDashboard to render the interactive map and MapMarker to represent selectable pothole detections. DetectionDetails will display the selected detection's image and metadata, while FilterManager will manage the user's filter selections. Dashboard requests will pass through PotholeAPI and PotholeService, with PotholeRepository and

DatabaseManager handling access to stored detection records. ImageStorageManager will manage access to the associated detection images.

DetectionReviewManager will handle approval and rejection decisions submitted through the review interface. Maintenance status updates and history retrieval will require corresponding backend operations to support FR-14 and the DetectionStatusHistory entity defined in Section 4.3.

The recording, upload, and processing-status screens will require UI controls and backend operations that connect collected footage and GPS data to VideoProcessor, GPSProcessor, PotholeDetector, and DetectionProcessor. These screens will display processing outcomes supplied by the backend. Their specific interface classes and processing-status operations remain to be defined in Section 4.2.

8. Version Control and Software Management System

Repository link: https://github.com/christopherthrasher/RoadWatch

9. Contributions of Team Members

Team Member	Activities Completed	Hours
Christopher Thrasher	Formatted document; wrote table of contents, sections 4.1, 4.2, 4.3	5
Victor Rosales	Sections 7.1-7.4	6
Vansh Sachdeva	Section 5, part of section 8	3.5
Noe Rios	Revised document, sections 2, 3, 8	4